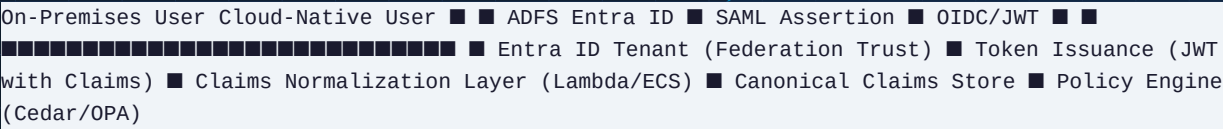


Enterprise identity environments are rarely clean. Most large organizations operate with a combination of Microsoft Entra ID (formerly Azure AD) as the modern cloud IDP and legacy ADFS for on-premises federation. Both systems produce identity tokens that must be normalized before they can be used for policy evaluation.

Enterprise Policy Interceptor Architecture for Agentic AI

1.1 Federation Architecture



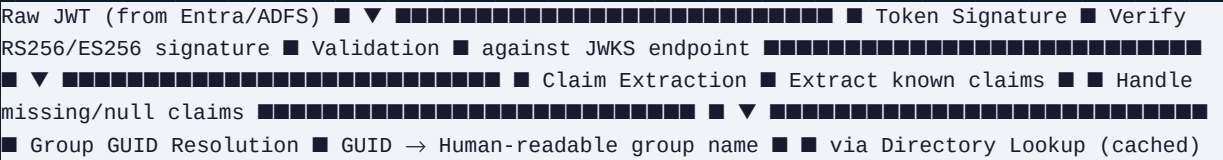
1.2 Token Types and Claims

Both ADFS and Entra ID issue tokens with different claim sets. The claims normalization layer must handle all variations and produce a consistent canonical representation:

Claim Type	ADFS Example	Entra ID Example	Canonical Form
User Principal	upn: john.smith@bank.com	upn: j.smith@bank.onmicrosoft.com	principal_id: john.smith
Department	department: Finance	extension_department: FINANCE	business_unit: FINANCE
Security Groups	groups: [GUID-1, GUID-2]	groups: [GUID-A, GUID-B]	roles: [payments_approver, trade_viewer]
Country/Region	I: United Kingdom	country: GB	geography: GB
Manager	manager: cn=mgr,dc=bank	manager: GUID-of-manager	manager_id: emp-42891
App Roles	roles: [FinanceAdmin]	roles: [Payments.Approve]	capabilities: [can_approve_payment]
Tenant	Not present	tid: TENANT-GUID	tenant_id: bank-prod
MFA	Not standard	amr: [mfa, pwd]	mfa_verified: true
Cost Center	costCenter: CC-4421	extension_costCenter: 4421	cost_center: CC-4421

Claims normalization is the most critical and most frequently under-engineered component of enterprise authorization. Without it, every policy is tightly coupled to the identity store implementation.

2.1 The Normalization Pipeline



[illegible]

2.2 Canonical Enterprise Claims Model

The canonical claims object is the contract between the identity layer and the policy engine. It must be stable, versioned, and independent of the underlying identity store:

```
[{"schema_version": "2.1", "principal": { "id": "emp-48291", "upn": "john.smith@bank.com", "display_name": "John Smith", "employee_type": "FULL_TIME" }, "organization": { "tenant_id": "bank-prod", "business_unit": "FINANCE", "department": "CAPITAL_MARKETS", "cost_center": "CC-4421", "geography": "GB", "legal_entity": "BANK_UK_LTD", "manager_id": "emp-12045" }, "capabilities": [ "can_approve_payment", "can_view_trade", "can_export_report" ], "context": { "mfa_verified": true, "session_age_minutes": 14, "auth_strength": "AAL2", "device_compliant": true, "network_zone": "CORPORATE", "risk_score": 12, "business_hours": true }, "issued_at": "2025-06-26T09:14:22Z", "expires_at": "2025-06-26T10:14:22Z" }
```

2.3 Role-to-Capability Mapping: Avoiding Group Coupling

The most important design decision in claims normalization is to never write Cedar or Rego policies that reference Entra group names or GUIDs directly. Instead, map identity-layer constructs to domain-level capabilities:

X ANTI-PATTERN

Anti-Pattern: `permit(principal, action, resource)` when `principal.groups` contains `'Payments_Admins_APAC_GBP'`;
— This policy breaks whenever the AD group is renamed, restructured, or the user moves region. The policy now encodes identity store implementation details.

✓ BEST PRACTICE

Best Practice: Map Entra groups to business capabilities in the normalization layer. The Cedar policy references capabilities only: `permit(principal, action::'approve_payment', resource)` when `principal.capabilities` contains `'can_approve_payment'`; — This policy is stable regardless of identity store changes.

Entra Group / Role	Maps To Capability	Policy Uses
Payments_Admns_APAC	can_approve_payment	principal.capabilities contains 'can_approve_payment'
Payments_Admns_EMEA	can_approve_payment	Same policy — geography context handles region
Finance_Readonly	can_view_financial_data	principal.capabilities contains 'can_view_financial_data'
DBA_Production	can_query_production_db	principal.capabilities contains 'can_query_production_db'

Entra Group / Role	Maps To Capability	Policy Uses
HR_Global_Admin	can_view_all_hr_records	principal.capabilities contains 'can_view_all_hr_records'
Compliance_Auditor	can_export_audit_log	principal.capabilities contains 'can_export_audit_log'

2.4 Large JWT Mitigation Strategies

- **JWT Trimming at PEP:** Strip claims not needed for authorization at the API Gateway before forwarding. Never pass megabyte JWTs to downstream services.
- **Capability Token Exchange:** Exchange the raw Entra JWT for a smaller, application-scoped capability token via the normalization service. RFC 8693 Token Exchange.
- **Reference Token Pattern:** Issue an opaque reference token to the client; the PEP introspects it at the normalization service to get the full claim set.
- **Claim Caching:** Cache normalized claim objects in ElastiCache (Redis) keyed by token hash. TTL aligned with token expiry. Reduces PIP lookups by >95%.
- **SCIM Attribute Push:** Use SCIM 2.0 to pre-populate a user attribute store (DynamoDB) rather than embedding all attributes in the JWT.

3. Cedar Policy Design

AWS Cedar is a purpose-built, formally verified policy language developed by Amazon for fine-grained authorization. Cedar policies are type-safe, decidable, and provably correct — properties that make it ideal for enterprise AI authorization.

3.1 Cedar Entity Schema

```
// Cedar Entity Schema (cedar-schema.json) { "namespaces": { "BankAI": { "entityTypes": {
  "User": { "memberOfTypes": ["Role"], "shape": { "type": "Record", "attributes": {
    "businessUnit": { "type": "String", "required": true }, "geography": { "type": "String",
    "required": true }, "mfaVerified": { "type": "Boolean", "required": true }, "riskScore": {
    "type": "Long", "required": false }, "capabilities": { "type": "Set", "element": { "type":
    "String" } } } } }, "Agent": { "shape": { "type": "Record", "attributes": { "agentType": {
    "type": "String" }, "delegatedFrom": { "type": "Entity", "name": "User" }, "confidenceScore": {
    "type": "Long" } } } }, "Tool": { "memberOfTypes": ["ToolGroup"], "shape": { "type": "Record",
    "attributes": { "dataClassification": { "type": "String" }, "requiresMFA": { "type": "Boolean"
    }, "allowedGeographies": { "type": "Set", "element": { "type": "String" } } } } }, "Resource":
    {}, "Role": {}, "ToolGroup": {} }, "actions": { "InvokeTool": { "appliesTo": {
    "principalTypes": ["User", "Agent"], "resourceTypes": ["Tool" ]}}, "ApprovePayment": {
    "appliesTo": { "principalTypes": ["User"], "resourceTypes": ["Resource" ]}}, "QueryDatabase": {
    "appliesTo": { "principalTypes": ["Agent"], "resourceTypes": ["Resource" ]} } } } }
```

3.2 Cedar Policy Patterns

Default Deny (Foundational)

```
// Every Cedar policy set must begin with an implicit default deny. // Cedar's default is deny –
policies only grant, never imply. // This policy makes the intent explicit for documentation:
forbid(principal, action, resource);
```

RBAC — Capability-Based Tool Access

```
// Tool invocation based on business capabilities (not AD groups) permit( principal, action ==
BankAI::Action::"InvokeTool", resource == BankAI::Tool::"PaymentApprovalTool" ) when {
principal.capabilities.contains("can_approve_payment") && principal.mfaVerified == true &&
context.businessHours == true };
```

ABAC — Geography-Scoped Authorization

```
// Finance users can only invoke HR tools for their own geography permit( principal, action ==
BankAI::Action::"InvokeTool", resource is BankAI::Tool ) when {
principal.capabilities.contains("can_view_hr_records") &&
resource.allowedGeographies.contains(principal.geography) };
```

ReBAC — Agent Delegation

```
// Agent may invoke tools ONLY within the scope of the delegating user permit( principal is
BankAI::Agent, action == BankAI::Action::"InvokeTool", resource ) when {
principal.delegatedFrom.capabilities.contains("can_approve_payment") &&
principal.delegatedFrom.mfaVerified == true && context.agentConfidenceScore >= 85 };
```

Temporal Policy — Business Hours Restriction

```
// Delete operations restricted to business hours forbid( principal, action ==
BankAI::Action::"InvokeTool", resource == BankAI::Tool::"DeleteRecordTool" ) when {
```

```
context.businessHours == false || context.riskScore > 70 };
```

Tenant Isolation

```
// Multi-tenant: principals can only access resources in their tenant forbid( principal,
action, resource ) when { resource has tenantId && principal has tenantId && resource.tenantId
!= principal.tenantId };
```

3.3 Cedar Policy Lifecycle Management

Phase	Activity	Tooling
Authoring	Policy authored in Cedar DSL	IDE plugins, cedar CLI, AVP console
Schema Validation	Type-check against entity schema	cedar validate --schema cedar-schema.json
Unit Testing	Test individual policies with test vectors	cedar test (test suite YAML)
Shadow Evaluation	Run new policy alongside existing — compare decisions	AVP Shadow Mode, decision log comparison
PR Review	Security team reviews policy changes as code	GitHub PR, automated cedar lint
Staging Deploy	Deploy to non-production AVP policy store	CI/CD pipeline (GitHub Actions/CodePipeline)
Production Deploy	Phased rollout with monitoring	CodeDeploy blue/green, CloudWatch alarms
Drift Detection	Alert on unauthorized policy changes	CloudTrail events on AVP, automated diff
Emergency Rollback	Revert to previous policy version	Git revert + pipeline trigger, < 5 min target

4. Rego / OPA Policy Design

Open Policy Agent (OPA) with the Rego policy language is the de facto standard for infrastructure policy — Kubernetes admission, Terraform validation, network policy, and cross-platform authorization. It is the ideal complement to Cedar in a hybrid architecture.

4.1 OPA Deployment Patterns

Pattern	Topology	Latency	Use Case
Embedded OPA	OPA Go library compiled into service	<0.5ms	Ultra-low-latency, single-service
OPA Sidecar	OPA container in same pod/task	0.5–2ms	Kubernetes/ECS per-service isolation
Central OPA Server	Shared OPA cluster (HA)	2–10ms network	Centralized policy, policy federation
OPA + Styra DAS	Managed OPA fleet + central management	2–10ms	Enterprise OPA at scale
OPA + WASM	Rego compiled to WASM, run in browser/edge	<1ms	Edge authorization, CDN enforcement

4.2 Rego Policy Examples

Kubernetes Admission — Require Non-Root

```
package kubernetes.admission.security deny[msg] { input.request.kind.kind == "Pod" container :=
input.request.object.spec.containers[_] container.securityContext.runAsNonRoot != true msg :=
sprintf("Container '%v' must run as non-root", [container.name]) }
```

Terraform Policy — Prohibit Public S3 Buckets

```
package terraform.aws.s3 deny[msg] { resource := input.resource_changes[_] resource.type ==
"aws_s3_bucket" resource.change.after.acl == "public-read" msg := sprintf("S3 bucket '%v' must
not be public", [resource.address]) }
```

API Authorization — Agent Tool Invocation

```
package agent.authorization import future.keywords.if import future.keywords.contains default
allow := false allow if { input.action == "invoke_tool" input.principal.capabilities[_] ==
required_capability input.context.risk_score < max_risk_threshold business_hours }
required_capability := cap if { tool_capability_map[input.resource.tool_name] == cap }
tool_capability_map := { "sql_query_tool": "can_query_database", "payment_tool":
"can_approve_payment", "hr_tool": "can_view_hr_records", } max_risk_threshold := 70
business_hours if { hour := time.clock(time.now_ns())[0] hour >= 8 hour < 18 }
```

Data Classification Policy

```
package data.classification # Deny access to PII data without DLP controls active deny[msg] if {
input.resource.classification == "PII" not input.context.dlp_active msg := "PII resource
requires active DLP controls" } # Deny export of SECRET data outside corporate network deny[msg]
if { input.resource.classification == "SECRET" input.action == "export"
input.context.network_zone != "CORPORATE" msg := "SECRET data cannot be exported from
```

```
non-corporate network" }
```

5. Cedar vs Rego: Decision Matrix

Cedar and Rego are complementary tools serving different domains. The choice between them should be driven by use case, not preference. The most mature enterprise architectures deploy both, with clear domain boundaries.

Dimension	AWS Cedar	OPA / Rego	Recommendation
Primary Domain	Application & user authorization	Infrastructure, K8s, cross-platform	Use both in appropriate domain
Policy Language	Cedar DSL — SQL-like, readable	Rego — Datalog-inspired, powerful	Cedar for business, Rego for infra
Type Safety	Strongly typed, schema-validated	Dynamically typed	Cedar for type-critical policies
Formal Verification	Yes — provably correct policies	No — runtime evaluation only	Cedar where correctness is critical
AWS Integration	Native (Amazon Verified Permissions)	Requires self-management or DAS	Cedar wins for AWS-native
Kubernetes	Not applicable	First-class (Gatekeeper/Konstraint)	OPA for K8s admission
Terraform/IaC	Not applicable	Excellent (OPA Conftest)	OPA for infrastructure policy
Ecosystem	AWS-specific, growing	Massive open-source ecosystem	OPA for cross-platform
Performance	Sub-millisecond (AVP managed)	1–5ms sidecar, <0.5ms embedded	Comparable for low-latency
Policy Management	AVP Console + CLI	Bundle server (S3) + Styra DAS	Styra DAS for enterprise OPA DAS
Decision Logging	Native to AVP	Requires configuration	Cedar easier for compliance logging
Multi-tenant	Native entity hierarchy	Custom implementation in Rego	Cedar simpler for multi-tenancy
WASM Support	No	Yes — compile to WASM	OPA for edge/CDN enforcement
Audit Trail	CloudTrail native integration	OPA decision logs → Elasticsearch	Cedar easier on AWS
Learning Curve	Low — business-readable	Medium — Datalog concepts	Cedar for business teams

6. Hybrid Cedar + Rego Architecture

The most effective enterprise architecture uses Cedar and OPA together, with clear domain ownership. This is not a compromise — it is an intentional design that leverages the strengths of each engine:

```
ENTERPRISE POLICY ARCHITECTURE ===== INFRASTRUCTURE DOMAIN
APPLICATION/AGENT DOMAIN ██████████ ████████████ OPA /
Rego AWS Cedar (Verified Permissions) ██████████ ████████████ .
Kubernetes admission • User authorization • Terraform validation • Agent permissions • Network
```


policy • Tool invocation control • Secret access policy • Resource access control • Docker image policy • Memory/knowledge access • Service mesh mTLS • Payment/trade approval • Compliance scanning • Data classification access • CI/CD pipeline gates • Multi-tenant isolation • Audit trail (CloudTrail) Policy Store: S3 Bundle Policy Store: AVP Policy Store Management: Styra DAS Management: AVP Console + IaC Deployment: GitOps Deployment: CI/CD + CodePipeline SHARED LAYER
■■■■■■■■■■■■■■■■■■■■ Claims Normalization Service (Canonical Claims → Both Engines) Policy Decision Aggregator (Combine Cedar + OPA decisions)

✓ BEST PRACTICE

Industry Evidence: Netflix uses OPA for infrastructure (Kubernetes, Spinnaker pipelines) and application-layer policy engines for content authorization. Capital One uses Cedar with AWS Verified Permissions for fine-grained banking application authorization. Uber uses OPA for microservice authorization across their heterogeneous stack. The hybrid model is the enterprise standard.

6.1 Decision Framework: When to Use Which Engine

Use Case	Recommended Engine	Rationale
Kubernetes Pod Security	OPA (Gatekeeper)	K8s-native, Gatekeeper is the standard
Terraform/CloudFormation policy gates	OPA (Conftest)	IaC scanning is OPA's strongest use case
User authorization to invoke an agent	Cedar (AVP)	User-resource model, AWS-native, audit trail
Agent permission to call a tool	Cedar (AVP)	Fine-grained, type-safe, traceable
Agent access to RAG knowledge	Cedar (AVP)	Document-level entity model, tenant isolation
Payment approval workflow	Cedar (AVP)	Formal verification, compliance audit trail
Network egress policy	OPA (Envoy integration)	L7 policy, service mesh native
CI/CD deployment gates	OPA (Conftest/Rego)	Pipeline integration, multi-cloud
Agent memory protection	Cedar (AVP)	Entity hierarchy, per-user scoping
Cross-cloud authorization	OPA	Cloud-agnostic, universal engine
Service-to-service mTLS policy	OPA (Istio/Envoy)	Service mesh integration standard
AI model output filtering	Cedar (AVP)	Structured output classification policy

